

Wishaw UI Backend API Reference

UI-facing API contract generated from the implemented backend controllers, DTOs, Spring Security rules, and the React service layer. All example paths use `/api` because the UI defaults to that base. The backend also exposes the same routes under `/api/v1`.

Generated

March 30, 2026

Canonical Base URL

`VITE_API_BASE_URL=/api`

UI Mock Switch

`VITE_USE MOCKS=false`

Auth Model

Bearer access token + refresh token

Integration Rules

Headers and Authentication

- Send Authorization: Bearer <accessToken> on every authenticated request.
- Use Content-Type: application/json for all write endpoints except photo upload.
- Photo upload uses multipart/form-data with a single file part named photo.
- The backend exposes X-Correlation-Id in responses for tracing.

Refresh and Error Handling

- If an authenticated call returns 401 with code: "TOKEN_EXPIRED", post the refresh token to `/api/auth/refresh` and retry once.
- All backend errors use the same JSON envelope shown below.
- The React client already expects error text from the HTTP body when present and falls back to HTTP {status}.

Canonical UI base: `/api`

Versioned alias: `/api/v1`

Public only: login, register, refresh

Admin-only writes called out below

Standard Error Envelope

```
{
  "timestamp": "2026-03-30T10:15:00Z",
  "status": 401,
  "error": "Authentication required",
  "code": "TOKEN_EXPIRED",
  "path": "/api/modules",
  "correlationId": "8d1d5600-5f92-4f8b-8f1b-1a3a933f0e3c",
}
```

```
"fieldErrors": {}  
}
```

Endpoint Matrix

The "UI Method" column maps directly to the methods in the React API client. Request and response schema names are detailed later in this document.

Auth and Session

	UI Method	Method	Path	Auth	Request	Response	Notes
	realApi.login	POST	/api/auth/login	Public	LoginRequest	AuthResponseDto	Returns access token, refresh token, user, and profile in one response.
	realApi.register	POST	/api/auth/register	Public	RegisterRequest	AuthResponseDto	Creates a player account by default.
	internal refresh	POST	/api/auth/refresh	Public	RefreshTokenRequest	RefreshTokenResponse	Called automatically by the UI after TOKEN_EXPIRED.
	realApi.logout	POST	/api/auth/logout	Token recommended	none	{ success: true }	Current backend path is public in security config, but the UI should still treat it as a session call.

| realApi.getMe
| GET
| /api/auth/me
| Bearer token
| none
| SessionDto
| Used during app bootstrap to restore session state.

Profile

|
| UI Method
| Method
| Path
| Auth
| Request
| Response
| Notes

| realApi.getProfile
| GET
| /api/profile
| Bearer token
| none
| ProfileDto
| Returns only the current user profile.

| realApi.updateProfile
| PUT
| /api/profile
| Bearer token
| ProfileDto
| ProfileDto
| Backend overwrites mutable profile fields from the supplied body.

| realApi.uploadPhoto
| POST
| /api/profile/photo
| Bearer token
| multipart/form-data with part photo
| { photoUrl: string }
| Current implementation synthesizes a data-URI avatar rather than storing the binary file.

| realApi.setOverlay
| POST
| /api/profile/photo/overlay
| Bearer token
| OverlayRequest
| { overlayTemplate: string }
| Empty or null template is normalized to an empty string in the response.

Catalogue, Modules, Badges, Centres

|

| UI Method

| Method

| Path

| Auth

| Request

| Response

| Notes

| realApi.getCentres

| GET

| /api/centres

| Bearer token

| none

| CentreDto[]

| Centre catalogue for registration and filtering.

| realApi.getGroups

| GET

| /api/groups?centreId={centreId}

| Bearer token

| centreId query optional

| GroupDto[]

| If centreId is omitted, all groups are returned.

| realApi.getMainBadges

| GET

| /api/badges/main

| Bearer token

| none

| MainBadgeDto[]

| Main badge catalogue.

| realApi.getSubBadges

| GET

| /api/badges/sub?moduleId={moduleId}

| Bearer token

| moduleId query optional

| SubBadgeDto[]

| Use the query to narrow sub-badges to one module.

| realApi.getUserBadgeProgress

| GET

| /api/badges/progress/{userId}

| Bearer token

| userId path

| UserBadgeProgressDto[]

| Returns progress bucketed by main badge.

| realApi.awardSubBadge

| POST
| /api/badges/award
| Admin or super admin
| BadgeAwardRequest
| { success: true }
| Awards one sub-badge to one user.

| realApi.getModules
| GET
| /api/modules
| Bearer token
| none
| ModuleDto[]
| Includes sub-badges and weekly schedule entries.

| realApi.getModule
| GET
| /api/modules/{id}
| Bearer token
| id path
| ModuleDto
| Module detail endpoint used by module detail screens.

Sports and Tournaments

|
| UI Method
| Method
| Path
| Auth
| Request
| Response
| Notes

| realApi.getSports
| GET
| /api/sports
| Bearer token
| none
| SportDto[]
| Read is open to any authenticated user.

| realApi.createSport
| POST
| /api/sports
| Admin or super admin
| SportUpsertRequest
| SportDto
| Admin management endpoint.

| realApi.updateSport

| PUT
| /api/sports/{id}
| Admin or super admin
| SportUpsertRequest
| SportDto
| Admin management endpoint.

| realApi.deleteSport
| DELETE
| /api/sports/{id}
| Admin or super admin
| none
| { success: true }
| Deletes by sport id.

| realApi.getTournaments
| GET
| /api/tournaments
| Bearer token
| none in backend
| TournamentListResponse
| The React client can send query filters, but the current backend ignores them.

| realApi.getTournament
| GET
| /api/tournaments/{id}
| Bearer token
| id path
| TournamentDto
| Tournament detail endpoint.

| realApi.createTournament
| POST
| /api/tournaments
| Admin or super admin
| TournamentUpsertRequest
| TournamentDto
| sportId is mandatory. Missing enum fields default to INDIVIDUAL, DRAFT, and points 3/1/0.

| realApi.updateTournament
| PUT
| /api/tournaments/{id}
| Admin or super admin
| TournamentUpsertRequest
| TournamentDto
| Same payload as create.

| realApi.publishTournament
| POST
| /api/tournaments/{id}/publish
| Admin or super admin
| none
| TournamentDto

| Sets status to PUBLISHED.

| realApi.cancelTournament

| POST

| /api/tournaments/{id}/cancel

| Admin or super admin

| none

| TournamentDto

| Sets status to CANCELLED.

| realApi.completeTournament

| POST

| /api/tournaments/{id}/complete

| Admin or super admin

| none

| TournamentDto

| Sets status to COMPLETED.

| realApi.joinTournament

| POST

| /api/tournaments/{id}/join

| Bearer token

| none

| { success: true }

| Idempotent. If already joined, backend still returns success.

| realApi.leaveTournament

| DELETE

| /api/tournaments/{id}/leave

| Bearer token

| none

| { success: true }

| Changes participant status to WITHDRAWN.

| realApi.getParticipants

| GET

| /api/tournaments/{id}/participants

| Bearer token

| id path

| ParticipantDto[]

| Ordered by repository id ascending.

Matches and Teams

|

| UI Method

| Method

| Path

| Auth

| Request

| Response

| Notes

| realApi.getMatches
| GET
| /api/matches/tournament/{id}
| Bearer token
| id path
| MatchDto[]
| Fetches all matches for one tournament.

| realApi.getMatch
| GET
| /api/matches/{id}
| Bearer token
| id path
| MatchDto
| Single match detail.

| realApi.createMatch
| POST
| /api/matches
| Admin or super admin
| MatchUpsertRequest
| MatchDto
| Participant list is recreated from the request body.

| realApi.updateMatch
| PUT
| /api/matches/{id}
| Admin or super admin
| MatchUpsertRequest
| MatchDto
| Same payload as create.

| realApi.submitScore
| POST
| /api/matches/{id}/score
| Admin or super admin
| ScoreSubmissionRequest
| { success: true }
| Also forces the match status to COMPLETED.

| realApi.getScore
| GET
| /api/matches/{id}/score
| Bearer token
| id path
| MatchScoreDto
| Used by match detail pages to read score data.

| realApi.getScoreAudit
| GET
| /api/matches/{id}/score/audit
| Bearer token
| id path

| Array<{ action, summary, at }>
| Currently emits one synthetic audit entry when a score exists.

| realApi.markAttendance
| POST
| /api/matches/{id}/attendance
| Admin or super admin
| AttendanceRequest
| { success: true }
| Attendance values must match the enum exactly.

| realApi.getTeams
| GET
| /api/teams/tournament/{id}
| Bearer token
| id path
| TeamDto[]
| Only teams for the requested tournament are returned.

| realApi.createTeam
| POST
| /api/teams
| Bearer token
| TeamCreateRequest
| TeamDto
| Current response starts with an empty members array.

Leaderboard, Calories, and Stats

|
| UI Method
| Method
| Path
| Auth
| Request
| Response
| Notes

| realApi.getLeaderboard
| GET
| /api/leaderboard/tournament/{id}
| Bearer token
| id path
| LeaderboardEntryDto[]
| Tournament-specific ranking.

| realApi.getGlobalLeaderboard
| GET
| /api/leaderboard/global
| Bearer token
| none

| LeaderboardEntryDto[]
| Cross-tournament leaderboard.

| realApi.getBadges
| GET
| /api/leaderboard/badges
| Bearer token
| none
| LtcBadgeDto[]
| Separate from the main/sub-badge learning catalogue.

| realApi.createBadge
| POST
| /api/leaderboard/badges
| Admin or super admin
| CreateBadgeRequest
| LtcBadgeDto
| Admin-managed leaderboard badge creation.

| realApi.assignBadge
| POST
| /api/leaderboard/badges/assign
| Bearer token
| BadgeAssignRequest
| { success: true }
| Not admin-gated in the current backend security rules.

| realApi.getUserBadges
| GET
| /api/leaderboard/badges/user/{userId}
| Bearer token
| userId path
| EarnedBadgeDto[]
| Leaderboard-earned badges for one user.

| realApi.logCalories
| POST
| /api/leaderboard/calories
| Bearer token
| CaloriesLogRequest
| { success: true }
| If userId is blank or null, the backend uses the current user.

| realApi.getUserCalories
| GET
| /api/leaderboard/calories/user/{userId}
| Bearer token
| userId path
| CaloriesSummaryDto
| Aggregated total plus per-sport breakdown.

| realApi.getPlayerStats
| GET

| /api/stats/player/{userId}
| Bearer token
| userId path
| PlayerStatsDto
| Player analytics and recent participation summary.

| realApi.getAdminDashboard
| GET
| /api/stats/admin/dashboard
| Admin or super admin
| none
| AdminDashboardDto
| Admin dashboard aggregate metrics.

Notifications and Sharing

|
| UI Method
| Method
| Path
| Auth
| Request
| Response
| Notes

| realApi.getNotifications
| GET
| /api/notifications
| Bearer token
| none
| NotificationListResponse
| Returns notifications plus unreadCount.

| realApi.markNotificationRead
| PUT
| /api/notifications/{id}/read
| Bearer token
| id path
| { success: true }
| Only the owner can mark a notification as read.

| realApi.markAllNotificationsRead
| PUT
| /api/notifications/read-all
| Bearer token
| none
| { success: true }
| Marks all notifications for the current user as read.

| realApi.createAnnouncement
| POST

| /api/notifications/announcements
| Bearer token
| AnnouncementRequest
| { success: true }
| Currently not admin-gated in security. Sends one announcement record to every user.

| realApi.getAnnouncements
| GET
| /api/notifications/announcements/tournament/{id}
| Bearer token
| id path
| AnnouncementDto[]
| Filtered to announcements linked to the tournament.

| realApi.getShareData
| GET
| /api/notifications/share/{type}/{id}
| Bearer token
| type, id path
| ShareDataDto
| Special handling exists only for type=tournament; any other type falls back to a generic home-page share payload.

| realApi.getGallery
| GET
| /api/notifications/gallery/tournament/{id}
| Bearer token
| id path
| []
| Current backend implementation validates the tournament id and returns an empty array.

Request Payload Examples

These are the JSON bodies the UI should send. Omit Content-Type: application/json only for the photo upload endpoint.

LoginRequest

```
{  
  "email": "player1@wishaw.local",  
  "password": "Password123!"  
}
```

RegisterRequest

```
{  
  "email": "new.player@wishaw.local",  
  "password": "Password123!",  
  "displayName": "New Player",  
  "firstName": "New",  
  "lastName": "Player"  
}
```

RefreshTokenRequest

```
{
```

```
"refreshToken": "eyJhbGciOiJIUzI1NiJ9.refresh-token"
}
```

ProfileDto

```
{
  "displayName": "Ava Singh",
  "firstName": "Ava",
  "lastName": "Singh",
  "bio": "Rocket League striker and mentor.",
  "photoUrl": "data:image/svg+xml;base64,...",
  "overlayTemplate": "winner-frame",
  "privacy": {
    "showInPublicList": true,
    "allowSocialSharing": true
  }
}
```

OverlayRequest

```
{
  "template": "winner-frame"
}
```

BadgeAwardRequest

```
{
  "userId": "u1",
  "subBadgeId": "sb7"
}
```

SportUpsertRequest

```
{
  "name": "Rocket League",
  "icon": "gamepad-2",
  "description": "Team-based football with cars.",
  "scoreFields": [
    { "key": "homeGoals", "label": "Home goals", "type": "number" },
    { "key": "awayGoals", "label": "Away goals", "type": "number" }
  ],
  "rankingPoints": {
    "win": 3,
    "draw": 1,
    "loss": 0
  }
}
```

TournamentUpsertRequest

```
{
  "name": "Spring Split Finals",
  "sportId": "s1",
  "description": "Centre-wide Rocket League finals.",
  "rules": "Best of three, standard ranked settings.",
  "venue": "Wishaw YMCA Arena",
  "type": "TEAM",
  "status": "DRAFT",
  "startDate": "2026-04-20T18:00:00Z",
  "endDate": "2026-04-20T21:00:00Z",
  "regStartDate": "2026-04-01T09:00:00Z",
  "regEndDate": "2026-04-18T23:59:59Z",
  "capacity": 16,
  "teamMinSize": 3,
  "teamMaxSize": 5,
  "pointsWin": 3,
  "pointsDraw": 1,
  "pointsLoss": 0
}
```

```
}
```

MatchUpsertRequest

```
{
  "tournamentId": "t1",
  "roundLabel": "Semi Final 1",
  "scheduledAt": "2026-04-20T18:30:00Z",
  "venue": "Stage A",
  "status": "SCHEDULED",
  "participants": [
    { "userId": "u1", "displayName": "Ava Singh", "attendance": "PRESENT" },
    { "userId": "u2", "displayName": "Noah Brown", "attendance": "PRESENT" }
  ]
}
```

ScoreSubmissionRequest

```
{
  "winnerId": "u1",
  "fields": {
    "home": { "goals": 3 },
    "away": { "goals": 1 }
  },
  "summary": "Ava Singh wins 3-1"
}
```

AttendanceRequest

```
{
  "records": [
    { "userId": "u1", "attendance": "PRESENT" },
    { "userId": "u2", "attendance": "LATE" }
  ]
}
```

TeamCreateRequest

```
{
  "name": "Wishaw Wolves",
  "tournamentId": "t1"
}
```

CreateBadgeRequest

```
{
  "name": "Fair Play",
  "icon": "shield-check",
  "description": "Awarded for consistent good sportsmanship."
}
```

BadgeAssignRequest

```
{
  "badgeId": "lb1",
  "userId": "u1"
}
```

CaloriesLogRequest

```
{
  "userId": "u1",
  "sportName": "Rocket League",
  "calories": 160
}
```

AnnouncementRequest

```
{
  "title": "Bracket published",
  "message": "Semi-final fixtures are now live.",
  "tournamentId": "t1"
}
```

Response Payload Samples

AuthResponseDto

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiJ9.access-token",
  "refreshToken": "eyJhbGciOiJIUzI1NiJ9.refresh-token",
  "user": {
    "id": "u1",
    "email": "player1@wishaw.local",
    "role": "PLAYER",
    "centreId": "c1"
  },
  "profile": {
    "displayName": "Ava Singh",
    "firstName": "Ava",
    "lastName": "Singh",
    "bio": "Rocket League striker and mentor.",
    "photoUrl": "data:image/svg+xml;base64,...",
    "overlayTemplate": "winner-frame",
    "privacy": {
      "showInPublicList": true,
      "allowSocialSharing": true
    }
  }
}
```

SessionDto

```
{
  "user": {
    "id": "u1",
    "email": "player1@wishaw.local",
    "role": "PLAYER",
    "centreId": "c1"
  },
  "profile": {
    "displayName": "Ava Singh",
    "firstName": "Ava",
    "lastName": "Singh",
    "bio": "Rocket League striker and mentor.",
    "photoUrl": "data:image/svg+xml;base64,...",
    "overlayTemplate": "winner-frame",
    "privacy": {
      "showInPublicList": true,
      "allowSocialSharing": true
    }
  }
}
```

ModuleDto

```
{
  "id": "m1",
  "name": "Rocket League Fundamentals",
  "game": "Rocket League",
  "description": "Introductory module covering teamwork and rotations.",
  "durationWeeks": 6,
  "subBadges": [
```

```

{
  "id": "sb1",
  "name": "Positioning Basics",
  "description": "Understand core positioning.",
  "mainBadgeId": "mb1",
  "mainBadgeName": "Game Mastery",
  "points": 10,
  "skills": ["Positioning", "Decision-making"],
  "moduleId": "m1"
}
],
"schedule": [
{
  "weekNo": 1,
  "focus": "Kickoff strategy",
  "subBadgeId": "sb1",
  "sessionPlanUrl": "/plans/m1-week1.pdf",
  "slidesUrl": "/slides/m1-week1.pdf"
}
],
"status": "ACTIVE"
}

```

SportDto

```

{
  "id": "s1",
  "name": "Rocket League",
  "icon": "gamepad-2",
  "description": "Team-based football with cars.",
  "scoreFields": [
    { "key": "homeGoals", "label": "Home goals", "type": "number" },
    { "key": "awayGoals", "label": "Away goals", "type": "number" }
  ],
  "rankingPoints": {
    "win": 3,
    "draw": 1,
    "loss": 0
  }
}

```

TournamentListResponse

```

{
  "tournaments": [
    {
      "id": "t1",
      "name": "Spring Split Finals",
      "sportId": "s1",
      "sport": {
        "id": "s1",
        "name": "Rocket League",
        "icon": "gamepad-2",
        "description": "Team-based football with cars.",
        "scoreFields": [
          { "key": "homeGoals", "label": "Home goals", "type": "number" }
        ],
        "rankingPoints": { "win": 3, "draw": 1, "loss": 0 }
      },
      "description": "Centre-wide Rocket League finals.",
      "rules": "Best of three",
      "venue": "Wishaw YMCA Arena",
      "type": "TEAM",
      "status": "PUBLISHED",
      "startDate": "2026-04-20T18:00:00Z",
      "endDate": "2026-04-20T21:00:00Z",
      "regStartDate": "2026-04-01T09:00:00Z",
      "regEndDate": "2026-04-18T23:59:59Z",
      "capacity": 16,
      "participantCount": 8,
      "teamMinSize": 3,
      "teamMaxSize": 5,
    }
  ]
}

```



```
"pointsWin": 3,
"pointsDraw": 1,
"pointsLoss": 0
}
],
"total": 1
}
```

MatchDto

```
{
  "id": "match1",
  "tournamentId": "t1",
  "roundLabel": "Semi Final 1",
  "scheduledAt": "2026-04-20T18:30:00Z",
  "venue": "Stage A",
  "status": "COMPLETED",
  "participants": [
    { "userId": "u1", "displayName": "Ava Singh", "attendance": "PRESENT" },
    { "userId": "u2", "displayName": "Noah Brown", "attendance": "PRESENT" }
  ],
  "score": {
    "winnerId": "u1",
    "fields": {
      "home": { "goals": 3 },
      "away": { "goals": 1 }
    }
  },
  "summary": "Ava Singh wins 3-1"
}
```

LeaderboardEntryDto

```
{
  "userId": "u1",
  "displayName": "Ava Singh",
  "centreId": "c1",
  "centreName": "Wishaw YMCA",
  "totalPoints": 75,
  "badgeLevels": {
    "Game Mastery": "Silver",
    "Teamwork": "Bronze"
  },
  "completedModules": 2
}
```

PlayerStatsDto

```
{
  "tournamentsJoined": 4,
  "activeTournaments": 1,
  "completedTournaments": 3,
  "matchesPlayed": 12,
  "wins": 8,
  "losses": 3,
  "draws": 1,
  "attendanceRate": 92,
  "badges": [
    { "id": "lb1", "name": "Fair Play", "icon": "shield-check", "earnedAt": "2026-03-12T10:15:00Z" }
  ],
  "tournaments": []
}
```

AdminDashboardDto

```
{
  "totalTournaments": 6,
  "activeTournaments": 2,
  "totalPlayers": 24,
  "totalMatches": 18,
}
```

```

"registrationsByTournament": [
{ "name": "Spring Split Finals", "count": 16 }
],
"attendanceTrend": [
{ "date": "2026-03-25", "rate": 89 }
],
"topPerformers": [
{ "displayName": "Ava Singh", "wins": 8 }
],
"recentScores": [
{ "matchLabel": "Semi Final 1", "score": "3-1", "time": "2026-04-20T18:30:00Z" }
]
}

```

NotificationListResponse

```

{
  "notifications": [
    {
      "id": "n1",
      "type": "ANNOUNCEMENT",
      "title": "Bracket published",
      "message": "Semi-final fixtures are now live.",
      "isRead": false,
      "createdAt": "2026-03-30T09:10:00Z",
      "linkTo": "/tournaments/t1"
    }
  ],
  "unreadCount": 1
}

```

ShareDataDto

```

{
  "title": "Spring Split Finals",
  "description": "Centre-wide Rocket League finals.",
  "url": "/tournaments/t1"
}

```

Enums and Integration Notes

Enums Used in Payloads

- UserRole: PLAYER, ADMIN, SUPER_ADMIN
- TournamentStatus: DRAFT, PUBLISHED, CANCELLED, COMPLETED
- TournamentType: INDIVIDUAL, TEAM
- ParticipantStatus: REGISTERED, WITHDRAWN
- AttendanceStatus: PRESENT, ABSENT, LATE, EXCUSED
- MatchStatus: SCHEDULED, IN_PROGRESS, COMPLETED, WALKOVER
- BadgeLevel: None, Bronze, Silver, Gold, Platinum

Observed Contract Gaps

- realApi.getTournaments(filters) can send query params, but the backend currently ignores them.
- getGallery is implemented as a tournament existence check plus an empty array.
- getShareData has special handling only for type=tournament.
- assignBadge and createAnnouncement are authenticated but not admin-gated in the current security config.

- uploadPhoto returns a generated data URI, not a persisted asset URL.

Recommended frontend dev configuration

Use `VITE_USE MOCKS=false` and point `VITE_API_BASE_URL` to `http://localhost:8080/api` when testing against the Spring Boot service locally.

Versioned routing

If you prefer the versioned contract, replace every `/api/...` example in this document with `/api/v1/...`. The backend maps both forms today.